elements from the corresponding rows in the first matrix, for a grand total of 1,010,000 elements. On the other hand, if they each handle a block of 100 elements by 100 elements (which is still 10,000 elements total), they need to access the values from 100 rows of the first matrix (100 x 1,000 = 100,000 elements) and 100 columns of the second matrix (another 100,000). This is only 200,000 elements, which is a five-fold reduction in the number of elements read. If you're reading fewer elements, there's less chance of a cache miss and the potential for greater performance.

It may therefore be better to divide the result matrix into small square or almost-square blocks rather than have each thread handle the entirety of a small number of rows. Of course, you can adjust the size of each block at runtime, depending on the size of the matrices and the available number of processors. As ever, if performance is important, it's vital to profile various options on the target architecture.

Chances are you're not doing matrix multiplication, so how does this apply to you? The same principles apply to any situation where you have large blocks of data to divide between threads; look at all the aspects of the data access patterns carefully, and identify the potential causes of performance hits. There may be similar circumstances in your problem domain where changing the division of work can improve performance without requiring any change to the basic algorithm.

OK, so we've looked at how access patterns in arrays can affect performance. What about other types of data structures?

8.3.2. Data access patterns in other data structures

Fundamentally, the same considerations apply when trying to optimize the data access patterns of other data structures as when optimizing access to arrays:

- Try to adjust the data distribution between threads so that data that's close together is worked on by the same thread.
- Try to minimize the data required by any given thread.